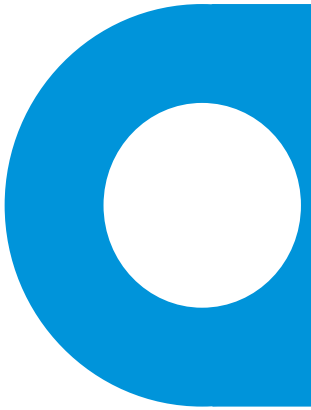


"Ss. Cyril and Methodius" University in Skopje
FACULTY OF COMPUTER
SCIENCE AND ENGINEERING



Collections

Advanced Programming

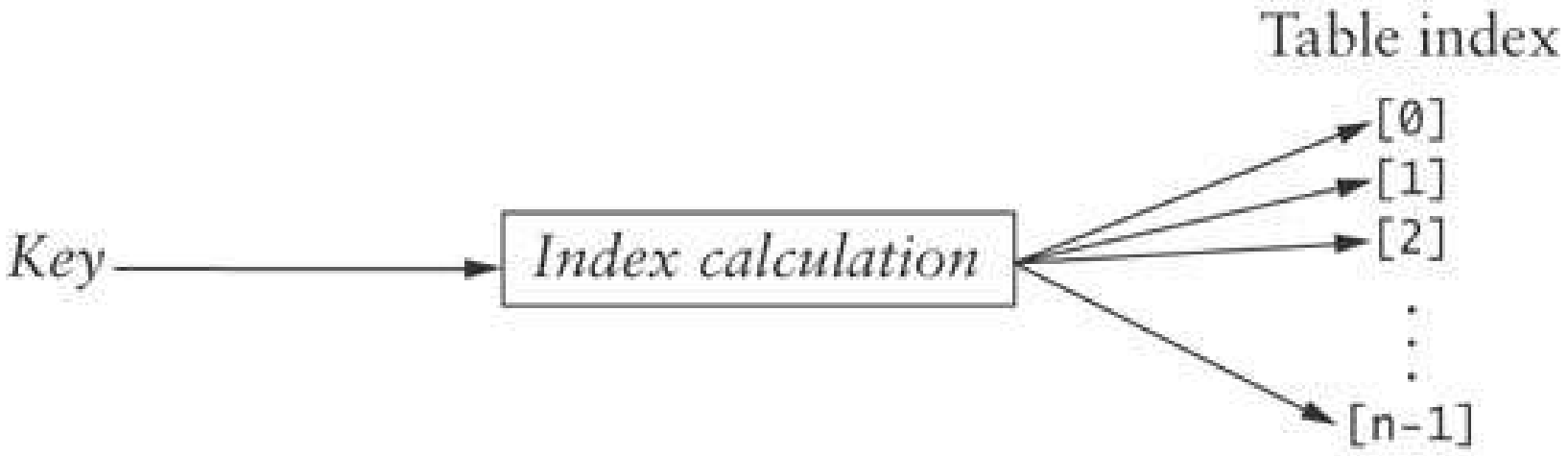
FCSE 2026

Hash tables

- The primary goal of hash tables is to provide **key-based access to values**, rather than location-based access.
- Imagine wanting to find a specific piece of data directly by its unique identifier (its key), instead of having to search through an entire list or array. Hash tables make this possible, allowing us to access elements in **constant time**, O(1).

Hash codes and index computation

- At its core, hashing involves transforming a key's value into an integer (its hash code). This hash code is then further transformed into an index within the hash table.



Hash codes and index computation

- The purpose of hash tables is to provide key-based access to the value, not by location
- Consider a scenario where we want to store and retrieve data based on character keys.
- If we're only dealing with ASCII characters (the first 128 Unicode values), we could use a simple table of size 128. In this case, each character's Unicode value could directly serve as its index in the table.

Hash codes and index computation

- But what if we need to support all 65,536 Unicode characters? Using a table of that size would be inefficient if we only typically use a small subset of characters.
- Let's say, on average, we expect to use about 100 characters. We could then use a hash table of a more manageable size, for instance, 200. The index for a character could be calculated using the modulo operator:

```
int index = unicode % 200
```

. . .	. . .
65	A, 8
66	B, 2
67	C, 3
68	D, 4
69	E, 12
70	F, 2
71	G, 2
72	H, 6
73	I, 7
74	J, 1
75	K, 2
. . .	. . .

Hash codes and index computation

- If the text contains:

```
. . . mañana (tomorrow), I'll finish my program. . .
```

- The following Unicode values are given:

Hexadecimal	Decimal	Name	Character
0x0029	41	right parenthesis	)
0x00F1	241	small letter n with tilde	ñ

- The indices of the letters 'ñ' and ')' are both 41

`41 % 200 = 41` and `241 % 200 = 41`

- This is called *collision;*

Methods for generating hash codes

- In most real-world applications, keys are not single characters but rather complex sequences of characters or digits (e.g., strings, IDs).
- The number of possible key values is typically far greater than the practical size of our hash table.
- Generating "good" hash codes is often an experimental process. The primary goal is to achieve a **random distribution of hash values** across the table.
- Simple algorithms, while easy to implement, can sometimes lead to a high number of collisions (where different keys produce the same hash code).

Java's String.hashCode() Explained

- **`String.hashCode()`** method calculates an integer hash value using the following formula:

$$s_0 \times 31^{(n-1)} + s_1 \times 31^{(n-2)} + \ldots + s_{n-1}$$

where:

  - s_i is the character at position i within the string.
  - n is the length of the string

- For example, the string "Cat" would have the following hash code:

$$\text{'C'} \times 31^2 + \text{'a'} \times 31 + \text{'t'} = 67 * 961 + 97 * 31 + 116 * 1 = 67{,}510$$

- The choice of 31 as a multiplier is significant. It's a prime number, and prime numbers tend to produce a relatively low number of collisions, leading to better distribution of hash values.

Understanding Hash Code Collisions

- Given the enormous number of possible strings, it's important to understand that the integer value returned by `String.hashCode()` cannot be guaranteed to be unique across all possible strings. However, the probability of a collision is generally low for well-designed hash functions.
- The probability of a collision occurring when mapping a hash code to an index (e.g., `s.hashCode()` % `table.length`) is **directly proportional to the load factor** of the hash table. A higher load factor, indicating a greater proportion of occupied slots relative to the table's total capacity, significantly increases the likelihood of different keys generating hash codes that map to the same table index.

Characteristics of a Good Hash Function

- A good hash function should possess two key characteristics:
  1. **Simplicity** and **Efficiency**: It must be relatively simple and efficient to compute. There's no benefit in using a hash function with O(n) complexity if its purpose is to avoid a search operation that also has O(n) time complexity. The goal is to make element access faster, not equally slow.
  2. **Even Distribution**: It should distribute hash codes as evenly as possible across the range of potential values to minimize collisions.

Methods for generating hash codes

- We will explore two primary ways to organize hash tables and handle collisions:

1. Open addressing
2. Chaining

- In open addressing, when we encounter a collision, we search for the next available slot in the table. One common method for this is *linear probing*.

Accessing an Item with Linear Probing

When searching for an element:

1. Calculate the hash code for the desired element to get an initial `index`.
2. Check the table at this `index`.
   - If the element at `index` has the same value (and therefore the same key), the element is found.
   - If the element at index has a different key or is `null`, it means we need to continue searching.
3. Increment the index by one (`index = (index + 1) % table.length;`) and repeat the check.
4. This incrementing continues until either:
   - The key is found.
   - A `null` slot is encountered (assuming the table isn't completely full, which would indicate the element is not present).

Open addressing

Algorithm for Accessing an Item in a Hash Table

1. Compute the index by taking the item's `hashCode() % table.length`.
2. `if table[index]` is `null`
3. The item is not in the table.
4. `else if table[index]` is equal to the item
5. The item is in the table.

   `else`
6. Continue to search the table by incrementing the index until either the item is found or a `null` entry is found.

Considerations for Open Addressing

- **Circular Behavior:** The table effectively acts like a circular list. When reaching the end, probing "wraps around" to the beginning.
- **Termination Condition:** To prevent infinite loops during searching (especially in a full table), the search must terminate. A common strategy is to stop when the current index for the next probe returns to the initial_index where the search began, without finding the key or an empty slot.
- **Load Factor:** Hash tables using open addressing are rarely allowed to become completely full. Their size is dynamically increased (rehashed) when the load factor (ratio of occupied slots to total slots) exceeds a certain threshold, ensuring performance.

Example of Hash Code Input

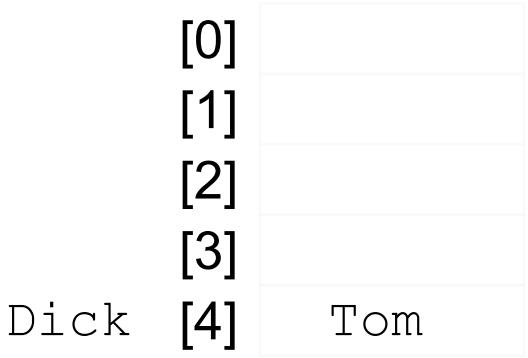
```
Tom  Dick  Harry  Sam  Pete
```

[0]	
[1]	
[2]	
[3]	
[4]	Tom

Name	hashCode()	hashCode()%5
"Tom"	84274	4
"Dick"	2129869	4
"Harry"	69496448	3
"Sam"	82879	4
"Pete"	2484038	3

Example of Hash Code Input

Dick Harry Sam Pete



Name	hashCode()	hashCode()%5
"Tom"	84274	4
"Dick"	2129869	4
"Harry"	69496448	3
"Sam"	82879	4
"Pete"	2484038	3

Example of Hash Code Input

Harry Sam Pete

	[0]	Dick
	[1]	
	[2]	
	[3]	
Dick	[4]	Tom

Name	hashCode()	hashCode()%5
"Tom"	84274	4
"Dick"	2129869	4
"Harry"	69496448	3
"Sam"	82879	4
"Pete"	2484038	3

Example of Hash Code Input

```
Harry  Sam  Pete
```

[0]	Dick
[1]	
[2]	
[3]	Harry
[4]	Tom

Name	hashCode()	hashCode()%5
"Tom"	84274	4
"Dick"	2129869	4
"Harry"	69496448	3
"Sam"	82879	4
"Pete"	2484038	3

Example of Hash Code Input

Sam Pete

	[0]	Dick
	[1]	
	[2]	
	[3]	Harry
Sam	[4]	Tom

Name	hashCode()	hashCode()%5
"Tom"	84274	4
"Dick"	2129869	4
"Harry"	69496448	3
"Sam"	82879	4
"Pete"	2484038	3

Example of Hash Code Input

Pete

Name	hashCode()	hashCode()%5
"Tom"	84274	4
"Dick"	2129869	4
"Harry"	69496448	3
"Sam"	82879	4
"Pete"	2484038	3

Sam	[0]	Dick
	[1]	
	[2]	
	[3]	Harry
Sam	[4]	Tom

Example of Hash Code Input

Pete

Name	hashCode()	hashCode()%5
"Tom"	84274	4
"Dick"	2129869	4
"Harry"	69496448	3
"Sam"	82879	4
"Pete"	2484038	3

Sam

[0]	Dick
[1]	Sam
[2]	
[3]	Harry
[4]	Tom

Example of Hash Code Input

Pete

Name	hashCode()	hashCode()%5
"Tom"	84274	4
"Dick"	2129869	4
"Harry"	69496448	3
"Sam"	82879	4
"Pete"	2484038	3

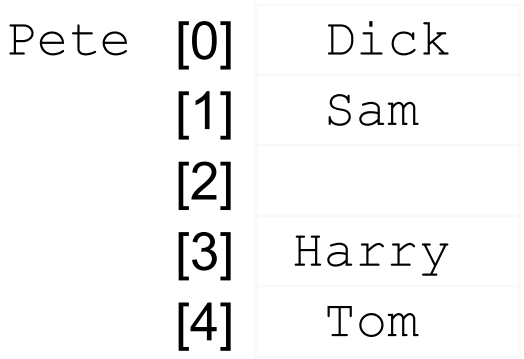
	[0]	Dick
	[1]	Sam
	[2]	
Pete	[3]	Harry
	[4]	Tom

Example of Hash Code Input

	[0]	Dick
	[1]	Sam
	[2]	
	[3]	Harry
Pete	[4]	Tom

Name	hashCode()	hashCode()%5
"Tom"	84274	4
"Dick"	2129869	4
"Harry"	69496448	3
"Sam"	82879	4
"Pete"	2484038	3

Example of Hash Code Input



Name	hashCode()	hashCode()%5
"Tom"	84274	4
"Dick"	2129869	4
"Harry"	69496448	3
"Sam"	82879	4
"Pete"	2484038	3

Example of Hash Code Input

	[0]	Dick
Pete	[1]	Sam
	[2]	
	[3]	Harry
	[4]	Tom

Name	hashCode()	hashCode()%5
"Tom"	84274	4
"Dick"	2129869	4
"Harry"	69496448	3
"Sam"	82879	4
"Pete"	2484038	3

Example of Hash Code Input

Name	hashCode()	hashCode()%5
"Tom"	84274	4
"Dick"	2129869	4
"Harry"	69496448	3
"Sam"	82879	4
"Pete"	2484038	3

	[0]	Dick
Pete	[1]	Sam
	[2]	Pete
	[3]	Harry
	[4]	Tom

The return of "Tom" or "Harry" is done in one step, O(1)

Because of collisions, returning the remaining elements requires a linear search

Example of Hash Code Input

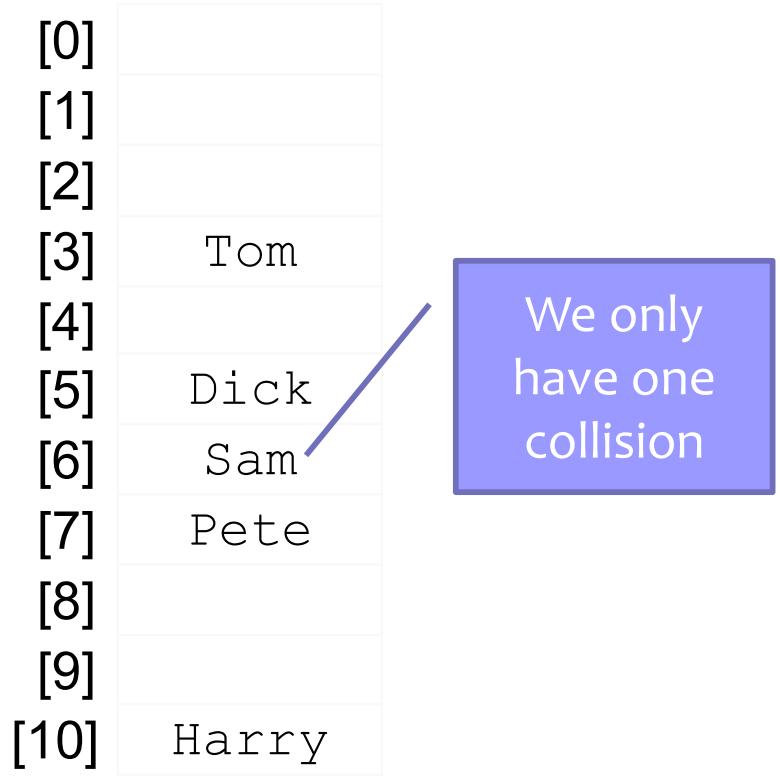
Name	hashCode()	hashCode()%11
"Tom"	84274	3
"Dick"	2129869	5
"Harry"	69496448	10
"Sam"	82879	5
"Pete"	2484038	7

[0]	
[1]	
[2]	
[3]	
[4]	
[5]	
[6]	
[7]	
[8]	
[9]	
[10]	

Example of Hash Code Input

Name	hashCode()	hashCode()%11
"Tom"	84274	3
"Dick"	2129869	5
"Harry"	69496448	10
"Sam"	82879	5
"Pete"	2484038	7

The best way to reduce the likelihood of collisions is to increase the size of the table



Iterating Through an Open Addressing Hash Table

- It's not meaningful to iterate through an open addressing hash table in a specific "order" because the element placement is essentially arbitrary. You can only iterate over the existing elements in their storage order.

[0]	Dick
[1]	Sam
[2]	Pete
[3]	Harry
[4]	Tom

Dick, Sam, Pete, Harry, Tom

[0]	
[1]	
[2]	
[3]	Tom
[4]	
[5]	Dick
[6]	Sam
[7]	Pete
[8]	
[9]	
[10]	Harry

Tom, Dick, Sam, Pete, Harry

Deleting an Element with Open Addressing

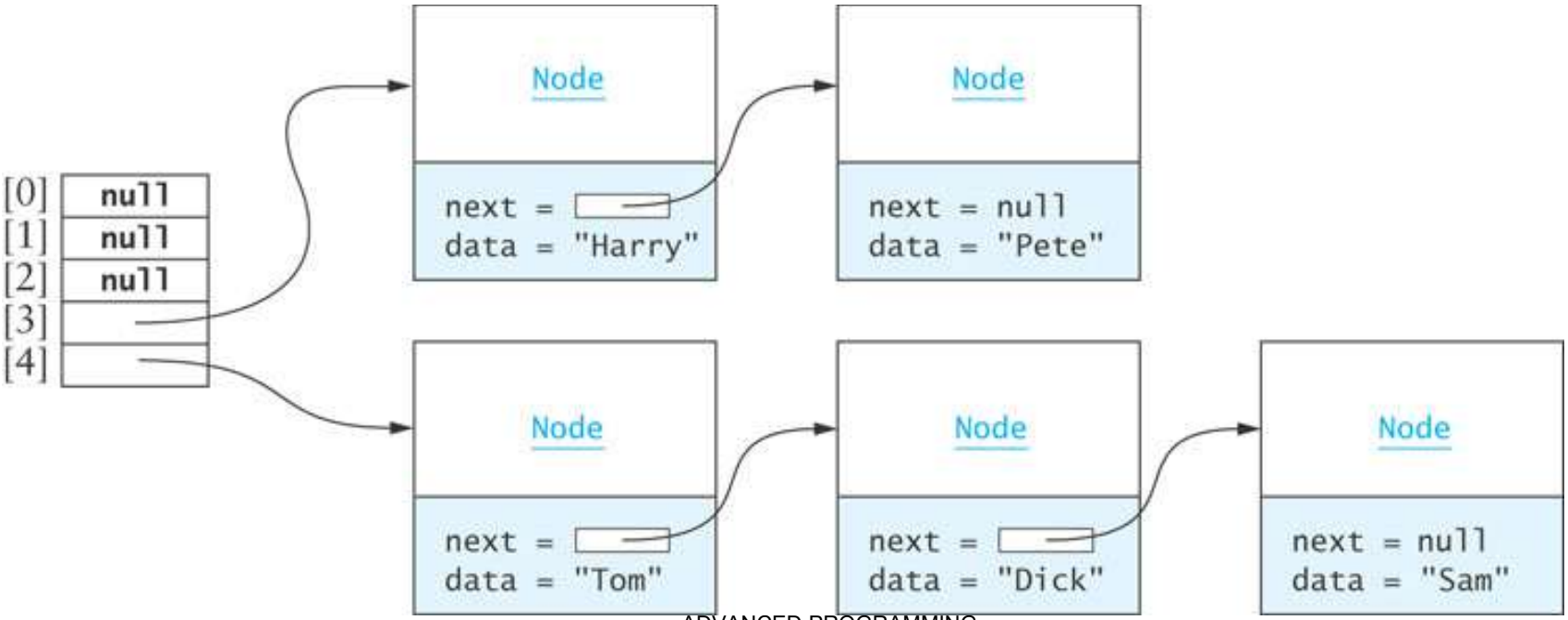
- Simply setting a slot to `null` after deletion in open addressing is problematic. If we delete an element that had collisions, subsequent searches for other elements that originally probed past the deleted element could incorrectly conclude that those elements don't exist (because they hit a `null` where a previously occupied slot used to be).
- Instead, the deleted location must be marked with a special predefined value (often called a "tombstone" or "deleted" marker) indicating that the slot was once occupied but is now logically empty. This tells subsequent searches to continue probing past this spot. However, frequent deletions can degrade performance by leaving many "tombstones" which increase search times.

Strategies to Reduce Collisions

- **Table Size**: Using a prime number for the hash table's size can help reduce the number of collisions by improving the distribution of indices.
- **Load Factor** and **Rehashing**: A more heavily populated table will inevitably result in a higher number of collisions and degraded performance. When the hash table's load factor exceeds a certain threshold, it should be **replaced with a new, larger table**. All existing elements must then be **rehashed** (re-calculate their hash codes and insert them into the new table). Deleted elements are *not* rehashed.

Chaining

- *Chaining* is an alternative to open addressing for resolving collisions.
- Instead of searching for the next empty slot in the main table, each entry (or "slot") in the hash table references a linked list (or sometimes another data structure like a tree). This linked list then holds all elements that map to the same hash code.
- This linked list is often referred to as a *bucket*. Consequently, this approach is sometimes called *bucket hashing*.



Chaining

Advantages of Chaining over Open Addressing:

- **Targeted Search:** When searching for an object, you only need to traverse the linked list (bucket) containing elements that share the same hash code, rather than potentially probing through many unrelated slots in the main table.
- **Flexible Capacity:** A chained hash table can store more elements than the number of available slots in the main table, as each bucket can grow as needed.
- **Simple Insertion:** If an element is determined not to exist within its designated bucket, it can simply be added to the beginning (or end) of that linked list.
- **Easy Deletion:** Elements can be removed directly from their respective linked lists without the need for special "tombstone" markers or complex logic to preserve search paths, as is required in open addressing.

Hash Table Performance

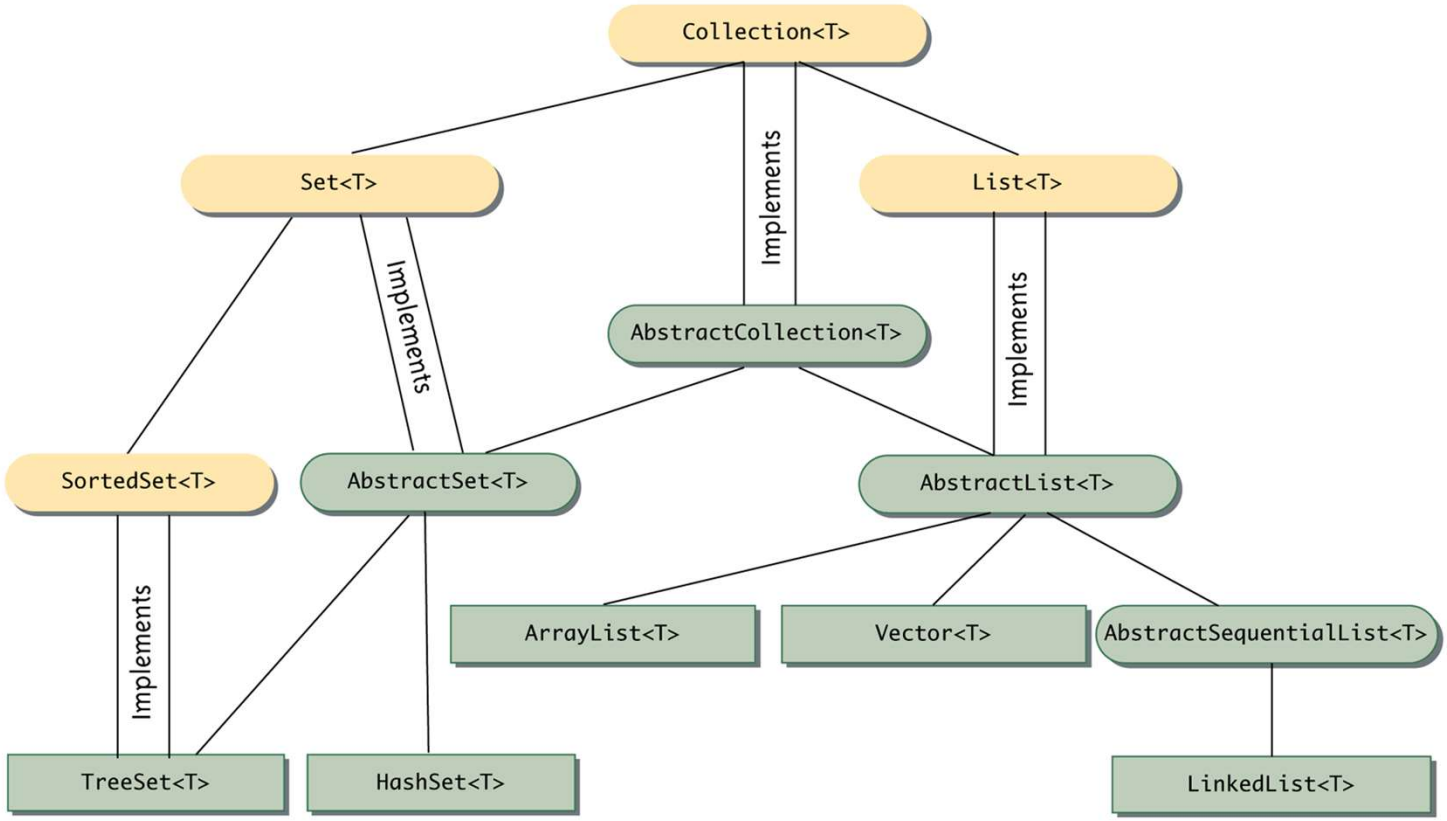
- The **load factor** is the percentage or ratio of occupied slots to the total capacity of the hash table. It has the most significant impact on hash table performance.
- A smaller load factor generally means better performance and a lower chance of collisions.
- If there are no collisions, the performance for searching and retrieving elements is O(1) (constant time), regardless of the table's size. This is the ideal scenario for hash tables.

Hash Table Performance vs. Sorted Arrays and Binary Search Trees

- **Binary Search (on a sorted array) and Binary Search Trees**: The number of comparisons required for these structures is O(log n).
  - For a sorted array with 128 elements, a binary search would require a maximum of 7 comparisons (since $2^7 = 128$). This is significantly more than a well-performing hash table, even one with a 90% load factor, which ideally aims for O(1) access.
  - Binary search trees exhibit similar logarithmic performance characteristics.
- Insertion and deletion

hash table	O(1) expected; in the worst case O(n)
sorted array	O(*n*)
Binary Search Tree	O(log *n*); in the worst case O(*n*)

Display 16.1 The Collection Landscape



A single line between two boxes means the lower class or interface is derived from (`extends`) the higher one.

T is a type parameter for the type of the elements stored in the collection.

`Set<T>` interface

- `Set<T>` is a collection that does not allow duplicate elements and has at most one `null` element
  - Adding "apples" to the set
    {"apples", "oranges", "pineapple"}
    results in the same set (no change)
- The operations typically performed on sets include:
  - **Membership Testing**: Checking if an element is present in the set.
  - **Adding Elements**: Incorporating a new element into the set.
  - **Removing Elements**: Deleting an existing element from the set.
  - **Union (A ∪ B)**: Combining elements from two sets.
  - **Intersection (A ∩ B)**: Finding common elements between two sets.
  - **Difference (A – B)**: Finding elements present in one set but not the other.
  - **Subset (A ⊂ B)**: Determining if one set's elements are all contained within another.

Set Operations

- **Union of two sets A and B:** The union results in a new set containing all elements that belong to A, or to B, or to both A and B.
  - Example: {1, 3, 5, 7} ∪ {2, 3, 4, 5} results in {1, 2, 3, 4, 5, 7}.
- **Intersection of two sets A and B:** The intersection results in a new set containing only those elements that belong to both A and B.
  - Example: {1, 3, 5, 7} ∩ {2, 3, 4, 5} results in {3, 5}.
- **Difference of two sets A and B (A – B):** The difference results in a new set containing only those elements that belong to A but not to B.
  - Example 1: {1, 3, 5, 7} – {2, 3, 4, 5} results in {1, 7}.
  - Example 2: {2, 3, 4, 5} – {1, 3, 5, 7} results in {2, 4}.
- **Subset (A ⊂ B):** Set A is a subset of set B if every element in A is also an element in B.
  - Example: {1, 3, 5, 7} ⊂ {1, 2, 3, 4, 5, 7} is true.

Set<T> interface methods (1 of 10)

Display 16.3 Methods in the Set<T> Interface

The Set<T> interface is in the `java.util` package.
The Set<T> interface extends the `Collection`<T> interface.
All the exception classes mentioned are the kind that are not required to be caught in a `catch` block or declared in a `throws` clause.
All the exception classes mentioned are in the package `java.lang` and so do not require any import statement.

CONSTRUCTORS

Although not officially required by the interface, any class that implements the Set<T> interface should have at least two constructors: a no-argument constructor that creates an empty Set<T> object, and a constructor with one parameter of type `Collection<? extends T>` that creates a Set<T> object with the same elements as the constructor argument.

(continued)

Set<T> interface methods (2 of 10)

Display 16.3 Methods in the Set<T> Interface

```
boolean isEmpty()
```

Returns `true` if the calling object is empty; otherwise returns `false`.

```
public boolean contains(Object target)
```

Returns `true` if the calling object contains at least one instance of `target`. Uses `target.equals` to determine if `target` is in the calling object.
Throws a `ClassCastException` if the type of `target` is incompatible with the calling object (optional).
Throws a `NullPointerException` if `target` is `null` and the calling object does not support `null` elements (optional).

(continued)

Set<T> interface methods (3 of 10)

Display 16.3 Methods in the Set<T> Interface

```
public boolean containsAll(Collection<?> collectionOfTargets)
```

Returns `true` if the calling object contains all of the elements in `collectionOfTargets`. For an `element` in `collectionOfTargets`, this method uses `element.equals` to determine if `element` is in the calling object. If `collectionOfTargets` is itself a `Set<T>`, this is a test to see if `collectionOfTargets` is a subset of the calling object.
Throws a `ClassCastException` if the types of one or more elements in `collectionOfTargets` are incompatible with the calling object (optional).
Throws a `NullPointerException` if `collectionOfTargets` contains one or more `null` elements and the calling object does not support `null` elements (optional).
Throws a `NullPointerException` if `collectionOfTargets` is `null`.

```
public boolean equals(Object other)
```

If the argument is a `Set<T>`, returns `true` if the calling object and the argument contain exactly the same elements; otherwise returns `false`. If the argument is not a `Set<T>`, `false` is returned.

(continued)

Set<T> interface methods (4 of 10)

Display 16.3 Methods in the Set<T> Interface

```
public int size()
```

Returns the number of elements in the calling object. If the calling object contains more than Integer.MAX_VALUE elements, returns Integer.MAX_VALUE.

```
Iterator<T> iterator()
```

Returns an iterator for the calling object. (Iterators are discussed in Section 16.2.)

```
public Object[] toArray()
```

Returns an array containing all of the elements in the calling object. A new array must be returned so that the calling object has no references to the returned array.

(continued)

Set<T> interface methods (5 of 10)

Display 16.3 Methods in the Set<T> Interface

```
public <E> E[] toArray(E[] a)
```

Note that the type parameter E is not the same as T. So, E can be any reference type; it need not be the type T in Collection<T>. For example, E might be an ancestor type of T.
Returns an array containing all of the elements in the calling object. The argument a is used primarily to specify the type of the array returned. The exact details are described in the table for the Collection<T> interface (Display 16.2).
Throws an ArrayStoreException if the type of a is not an ancestor type of the type of every element in the calling object.
Throws a NullPointerException if a is null.

```
public int hashCode()
```

Returns the hash code value for the calling object. Neither hash codes nor this method are discussed in this book. This entry is here only to make the definition of the Set<T> interface complete. You can safely ignore this entry until you go on to study hash codes in a more advanced book. In the meantime, if you need to implement this method, have it throw an UnsupportedOperationException.

(continued)

Set<T> interface methods (6 of 10)

Display 16.3 Methods in the Set<T> Interface

ADDING AND REMOVING ELEMENTS

As with the Collection<T> interface, the following methods are optional, which means they still must be implemented, but the implementation can simply throw an UnsupportedOperationException if for some reason you do not want to give them a "real" implementation. An UnsupportedOperationException is a RunTimeException and so is not required to be caught or declared in a throws clause.

```
public boolean add(T element) (Optional)
```

If element is not already in the calling object, element is added to the calling object and true is returned. If element is in the calling object, the calling object is unchanged and false is returned.
Throws an UnsupportedOperationException if this method is not supported by the class that implements this interface.
Throws a ClassCastException if the class of element prevents it from being added to the set.
Throws a NullPointerException if element is null and the set does not support null elements.
Throws an IllegalArgumentException if some other aspect of element prevents it from being added to this set.

(continued)

Set<T> interface methods (7 of 10)

Display 16.3 Methods in the Set<T> Interface

```
public boolean addAll(Collection<? extends T> collectionToAdd) (Optional)
```

Ensures that the calling object contains all the elements in `collectionToAdd`. Returns `true` if the calling object changed as a result of the call; returns `false` otherwise. Thus, if `collectionToAdd` is a `Set<T>`, then the calling object is changed to the union of itself with `collectionToAdd`.
Throws an `UnsupportedOperationException` if this method is not supported by the class that implements this interface.
Throws a `ClassCastException` if the class of some element of `collectionToAdd` prevents it from being added to the calling object.
Throws a `NullPointerException` if `collectionToAdd` contains one or more `null` elements and the calling object does not support `null` elements, or if `collectionToAdd` is `null`.
Throws an `IllegalArgumentException` if some aspect of some element of `collectionToAdd` prevents it from being added to the calling object.

(continued)

Set<T> interface methods (8 of 10)

Display 16.3 Methods in the Set<T> Interface

```
public boolean remove(Object element) (Optional)
```

Removes the `element` from the calling object, if it is present. Returns `true` if the calling object contained the `element`; returns `false` otherwise.
Throws an `UnsupportedOperationException` if this method is not supported by the class that implements this interface.
Throws a `ClassCastException` if the type of `element` is incompatible with the calling object (optional).
Throws a `NullPointerException` if `element` is `null` and the calling object does not support `null` elements (optional).

(continued)

Set<T> interface methods (9 of 10)

Display 16.3 Methods in the Set<T> Interface

```
public boolean removeAll(Collection<?> collectionToRemove) (Optional)
```

Removes all the calling object's elements that are also contained in `collectionToRemove`. Returns `true` if the calling object was changed; otherwise returns `false`.
Throws an `UnsupportedOperationException` if this method is not supported by the class that implements this interface.
Throws a `ClassCastException` if the types of one or more elements in `collectionToRemove` are incompatible with the calling object (optional).
Throws a `NullPointerException` if the calling object contains a `null` element and `collectionToRemove` does not support `null` elements (optional).
Throws a `NullPointerException` if `collectionToRemove` is `null`.

(continued)

Set<T> interface methods (10 of 10)

Display 16.3 Methods in the Set<T> Interface

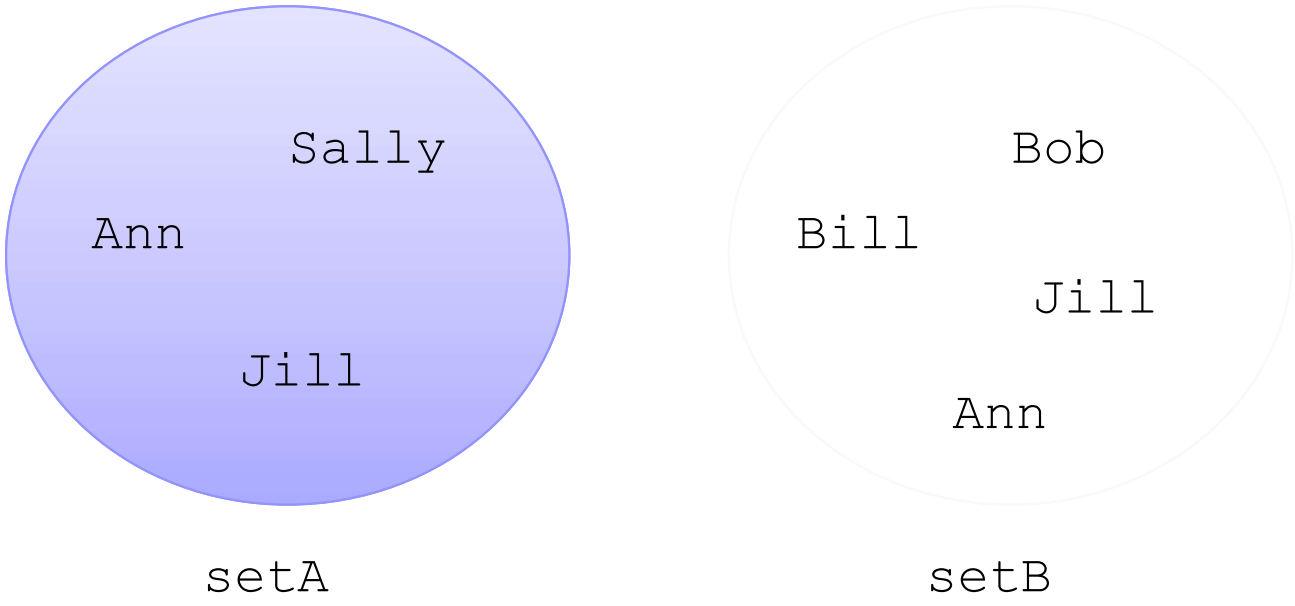
`public void clear()` *(Optional)*

Removes all the elements from the calling object.
Throws an `UnsupportedOperationException` if this method is not supported by the class that implements this interface.

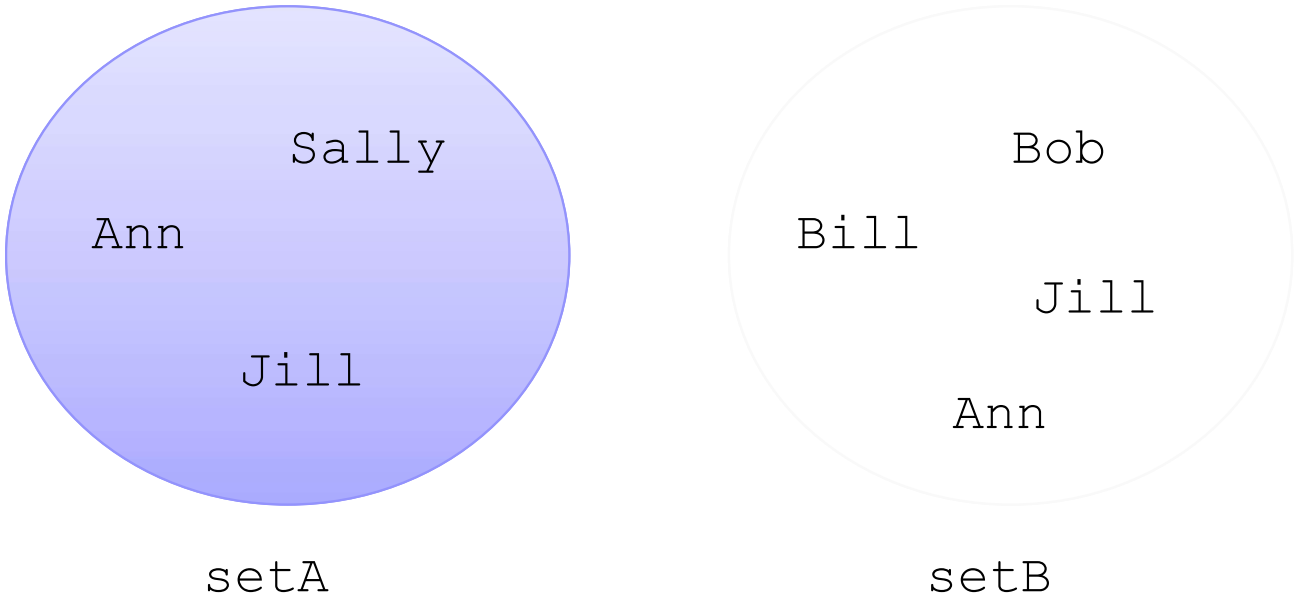
`public boolean retainAll(Collection<?> saveElements)` *(Optional)*

Retains only the elements in the calling object that are also contained in the collection `saveElements`. In other words, removes from the calling object all of its elements that are not contained in the collection `saveElements`. Returns `true` if the calling object was changed; otherwise returns `false`. If the argument is itself a `Set<T>`, this changes the calling object to the intersection of itself with the argument.
Throws an `UnsupportedOperationException` if this method is not supported by the class that implements this interface.
Throws a `ClassCastException` if the types of one or more elements in the calling object are incompatible with `saveElements` (optional).
Throws a `NullPointerException` if `saveElements` contains a `null` element and the calling object does not support `null` elements (optional).
Throws a `NullPointerException` if `saveElements` is `null`.

`Set<T>` interface examples

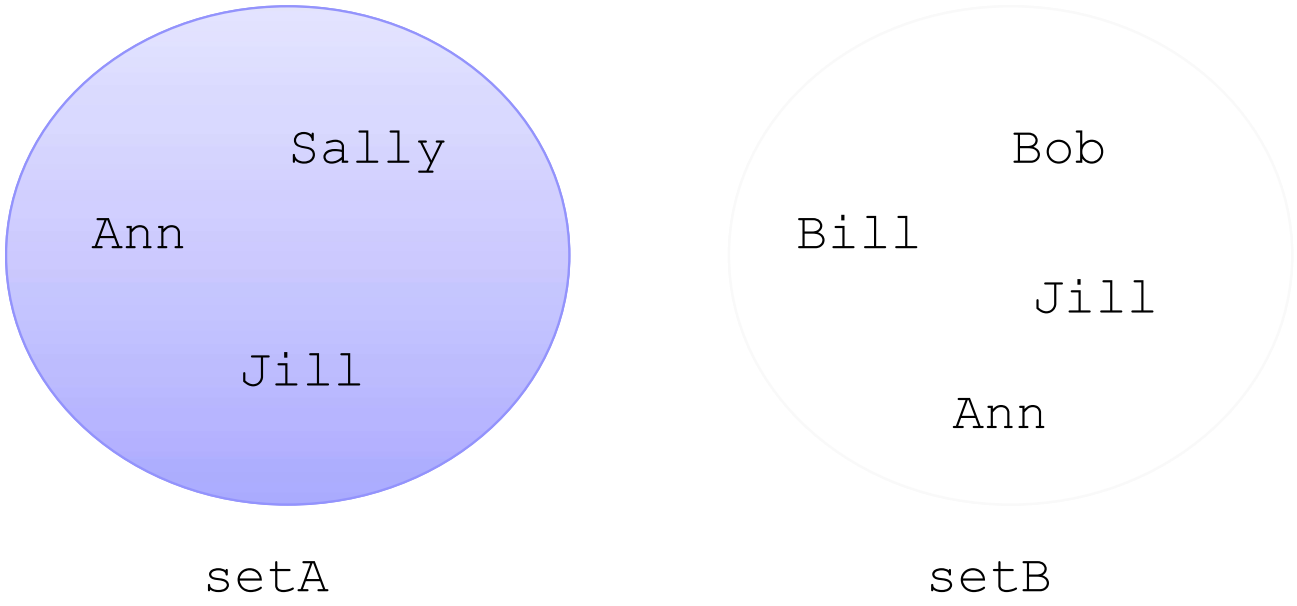


`Set<T>` interface examples




```
setA.addAll(setB);
```

Set<T> interface examples

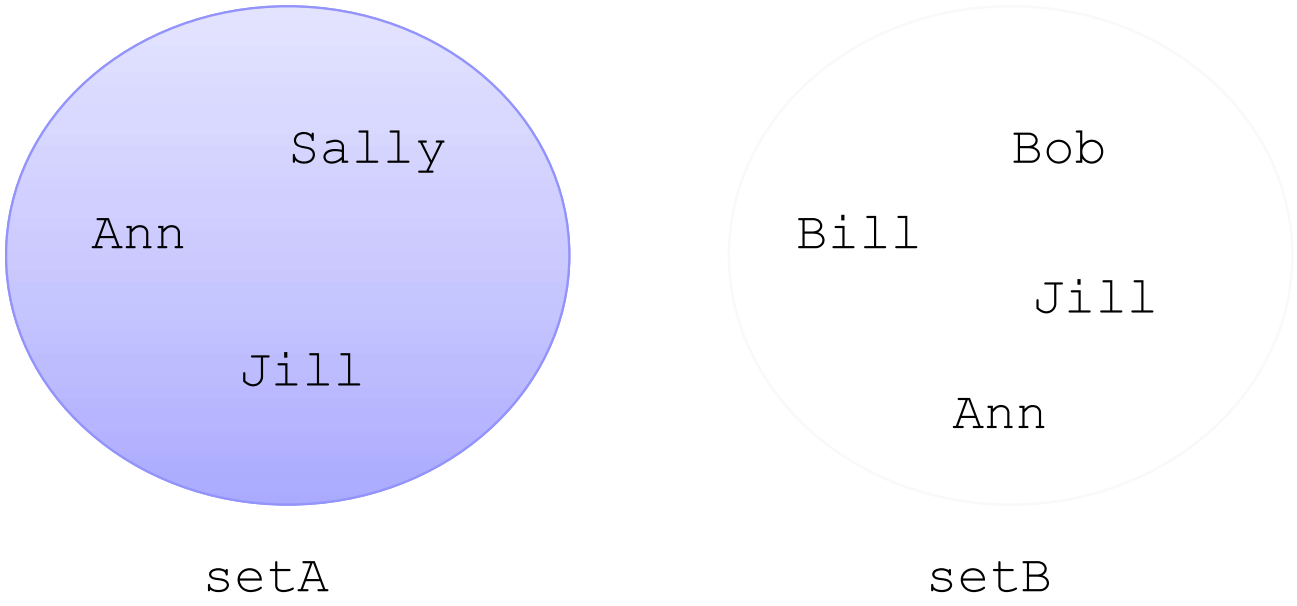



```
setA.addAll(setB);

System.out.println(setA);
```

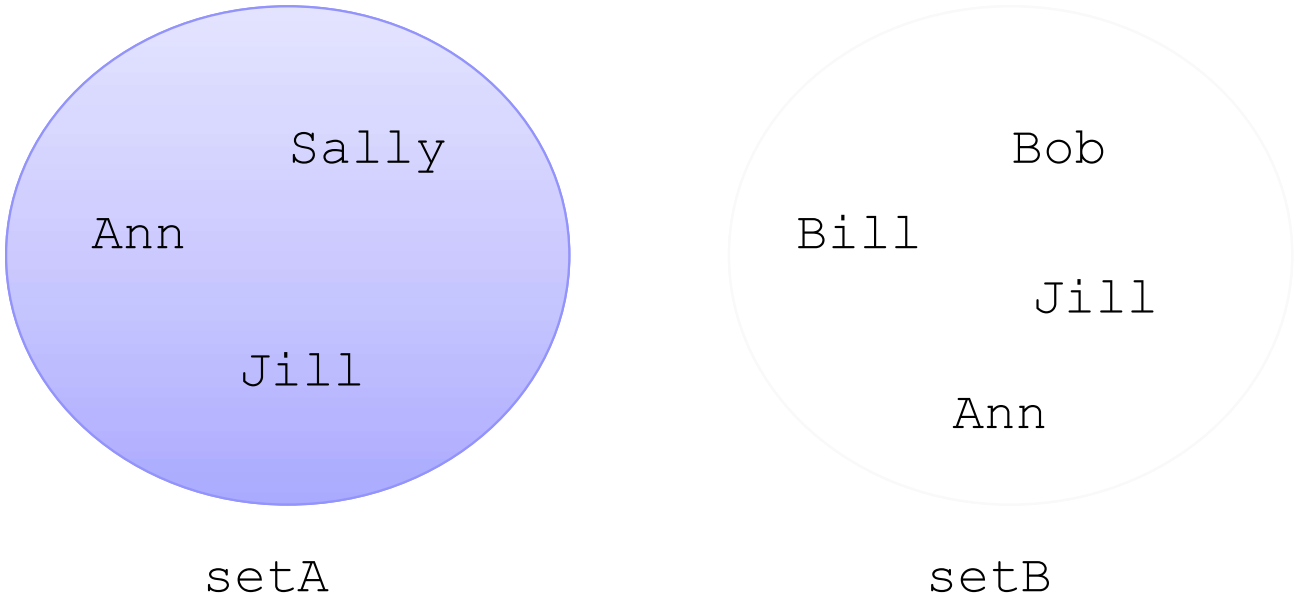
Output:
[Bill, Jill, Ann, Sally, Bob]

`Set<T>` interface examples



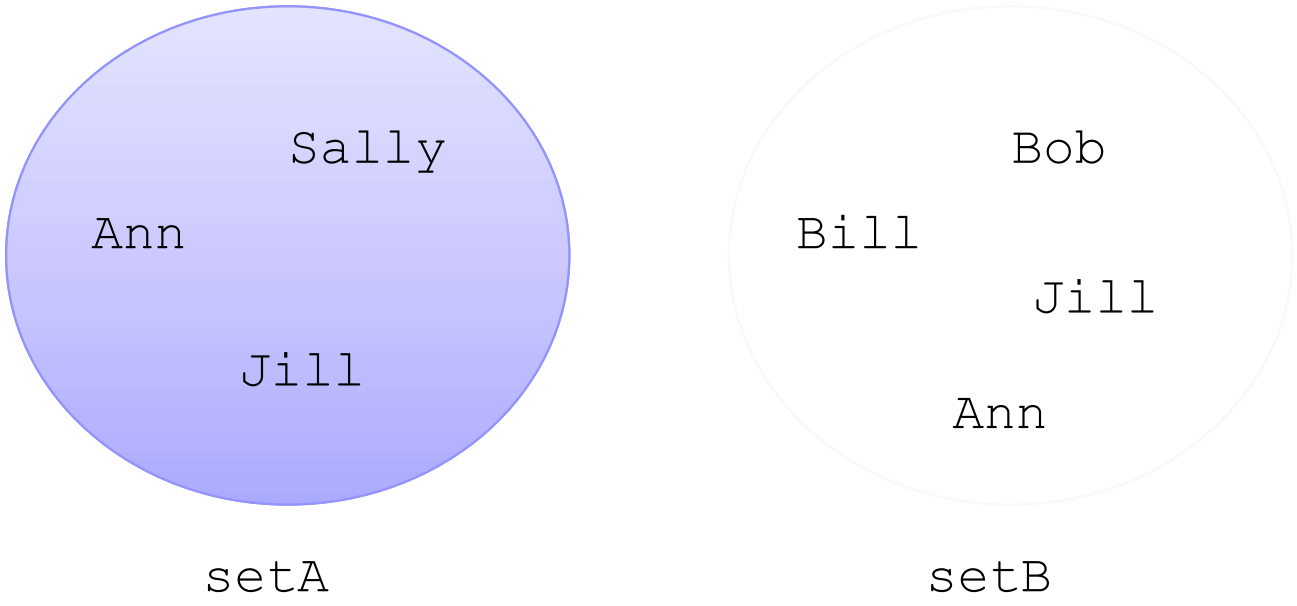
If a copy of the original **`setA`** set is the **`setACopy`** set, then . . .

`Set<T>` interface examples




```
setACopy.retainAll(setB);
```

`Set<T>` interface examples

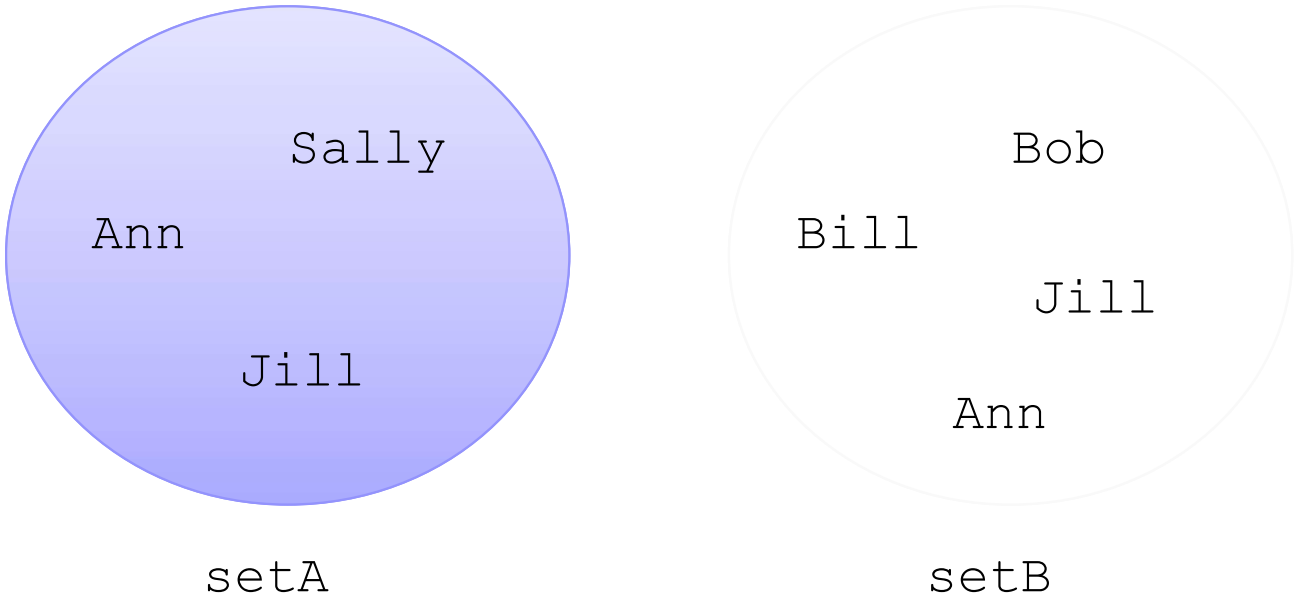



```
setACopy.retainAll(setB);

System.out.println(setACopy);
```

Output:
[Jill, Ann]

`Set<T>` interface examples




```
setACopy.removeAll(setB);

System.out.println(setACopy);
```

Output:
[Sally]

`List` vs. `Set` - Key Differences

- The **`Set`** Interface:
  - Collections that implement the **`Set`** interface are designed to **contain unique elements only**. This means a **`Set`** will never store duplicate objects.
  - Unlike the **`List.add()`** method, which always adds an element and returns true, the **`Set.add()`** method returns false if you attempt to insert an element that is already present in the set (i.e., a duplicate). If the element is successfully added, it returns true.
  - Crucially, **`Set<T>`** does not have a **`get(index)`** method. Therefore, elements cannot be accessed by their numerical position or index, as the concept of *order is not inherent* to a Set.
- The **`List`** Interface (for comparison):
  - List collections, on the other hand, **allow duplicate** elements. You can add the same object multiple times to a List.
  - List **maintains the insertion order** of its elements.
  - List provides a **`get(index)`** method, allowing elements to be accessed based on their zero-based numerical position.
- In summary, use a **`Set`** when you need a collection of unique items where the order of elements doesn't matter, and a **`List`** when you need an ordered collection that can contain duplicate elements.

Java Set Implementations: `HashSet` vs. `TreeSet`

```
public class HashSet<E> extends AbstractSet<E>
implements Set<E>, Cloneable, Serializable
```

- This class is a concrete implementation of the `Set<E>` interface, powered by a hash table.
- HashSet **does not guarantee any specific order** for its elements. Furthermore, it does not guarantee that the order will remain constant over time (e.g., after a rehash).
- It permits the insertion of a **single null element**.

HashSet<T>

- HashSet provides **constant-time (O(1)) performance** for its basic operations: `add`, `remove`, `contains`, and `size`. This optimal performance assumes that the hash function effectively distributes elements uniformly across its buckets (hash table slots).
- **Iteration Time:** The time complexity for iterating over a HashSet is proportional to the size of the set (the number of elements) plus the "capacity" of its buckets (the number of slots in the hash table).
- **Initial Capacity and Load Factor:** It's important not to set the initial capacity excessively large (or the load factor too small) if iteration performance is a critical concern, as this can lead to wasted memory and slower iteration.
  - By default, HashSet has an initial capacity of **16** and a load factor of **0.75**. These values are chosen to balance memory usage and performance.

TreeSet<T>

```
public class TreeSet<E> extends AbstractSet<E>
implements NavigableSet<E>, Cloneable, Serializable
```

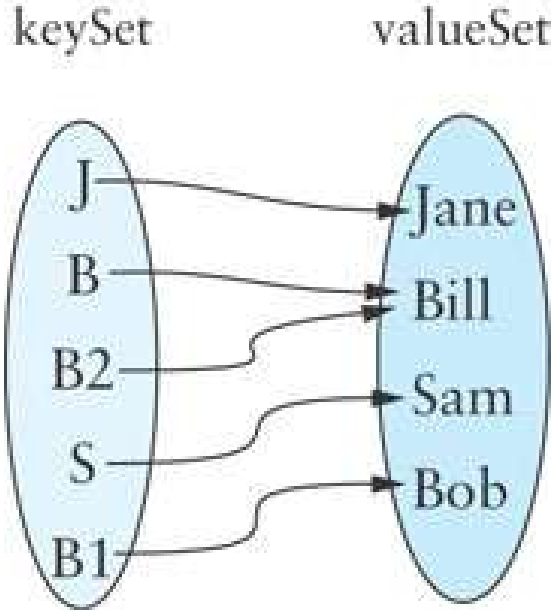
- This class is another concrete implementation of the `Set<E>` interface (and also `NavigableSet<E>`), but it stores its elements in a **sorted order**.
- Elements in a `TreeSet` are ordered either according to their **natural ordering** (by implementing the `Comparable` interface and its `compareTo` method) or by a custom `Comparator` provided when the set's constructor is called.
- `TreeSet` **does *not* allow null** elements.
- `TreeSet` guarantees **logarithmic time complexity (O(log n))** for its basic operations: `add`, `remove`, and `contains`. This is due to its underlying data structure, typically a Red-Black Tree (a self-balancing binary search tree).
- It's crucial to ensure that the `equals` method of the elements stored in a `TreeSet` is consistent with the `compareTo` method (or the provided `Comparator`).
  - If `x.equals(y)` is true, then `x.compareTo(y)` (or `comparator.compare(x, y)`) should return 0. Inconsistent behavior can lead to unexpected results.

ДЕМО

Maps and the **Map<K,V>** interface

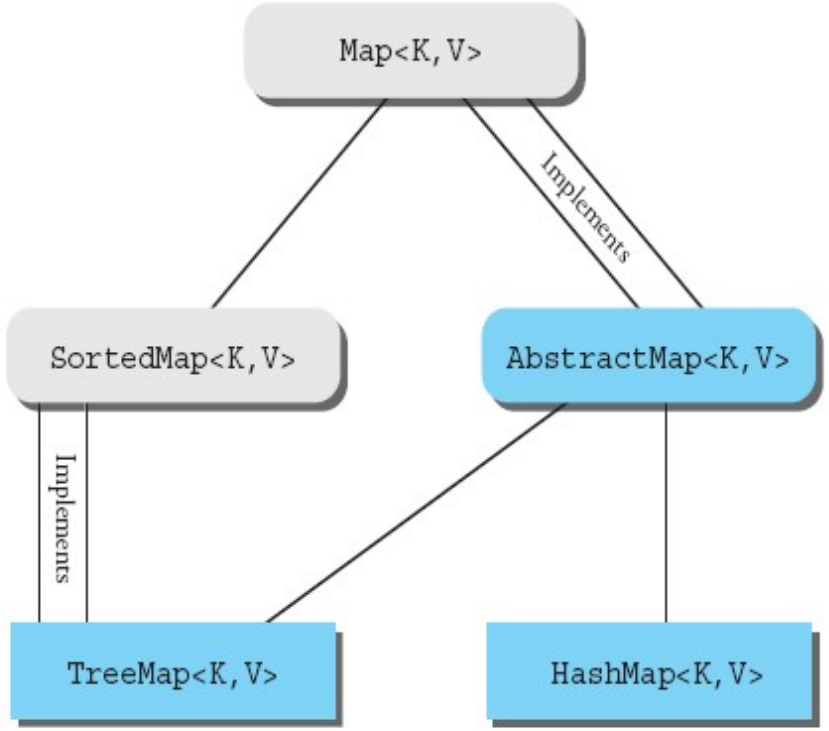
The `Map<K,V>` interface in Java is closely related to the `Set<T>` interface.

- **Mathematical Concept:** Mathematically, a map (also known as a dictionary or associative array) is a **collection of key-value pairs**.
- **Key Uniqueness:** The **keys must be unique**, meaning each key can map to at most one value. However, **values do not need to be unique**; multiple keys can map to the same value.
- **Efficient Storage:** A Map provides an efficient way to store and manipulate information in a tabular format, allowing for quick retrieval of values based on their associated keys.
- **Many-to-One Mapping:** A map can indeed have a "many-to-one" mapping, where different keys point to the same value.
  - *Example:* (B, Bill), (B2, Bill) – here, both B and B2 are unique keys, but they both map to the value Bill.




```
{(J, Jane), (B, Bill),
 (S, Sam), (B1, Bob),
 (B2,  Bill)}
```

Display 16.8 The Map Landscape



A single line between two boxes means the lower class or interface is derived from (extends) the higher one.

K and V are type parameters for the type of the keys and elements stored in the map.

`Map<K,V>` interface methods (1 of 4)

Display 16.9 Method Headings in the `Map<K,V>` Interface (part 1 of 3)

The `Map<K,V>` interface is in the `java.util` package.

CONSTRUCTORS

Although not officially required by the interface, any class that implements the `Map<K,V>` interface should have at least two constructors: a no-argument constructor that creates an empty `Map<K,V>` object, and a constructor with one `Map<K,V>` parameter that creates a `Map<K,V>` object with the same elements as the constructor argument. The interface does not specify whether the copy produced by the one-argument constructor is a shallow copy or a deep copy of its argument.

`Map<K,V>` interface methods (2 of 4)

Display 16.9 Method Headings in the `Map<K,V>` Interface (part 2 of 3)

METHODS
`boolean isEmpty()` Returns `true` if the calling object is empty; otherwise returns `false`.
`public boolean containsValue(Object value)` Returns `true` if the calling object contains at least one or more keys that map to an instance of `value`.
`public boolean containsKey(Object key)` Returns `true` if the calling object contains `key` as one of its keys.
`public boolean equals(Object other)` This is the `equals` of the map, not the `equals` of the elements in the map. Overrides the inherited method `equals`.
`public int size()` Returns the number of (key, value) mappings in the calling object.

`Map<K,V>` interface methods (3 of 4)

```
public int hashCode()
```

Returns the hash code value for the calling object.

```
public Set<Map.Entry<K,V>> entrySet()
```

Returns a set *view* consisting of (key, value) mappings for all entries in the map. Changes to the map are reflected in the set and vice versa.

```
public Collection<V> values()
```

Returns a collection *view* consisting of all values in the map. Changes to the map are reflected in the collection and vice versa.

```
public V get(Object key)
```

Returns the value to which the calling object maps `key`. If `key` is not in the map, then `null` is returned. Note that this does not always mean that the key is not in the map because it is possible to map a key to `null`. The `containsKey` method can be used to distinguish the two cases.

`Map<K,V>` interface methods (4 of 4)

OPTIONAL METHODS

The following methods are optional, which means they still must be implemented, but the implementation can simply throw an `UnsupportedOperationException` if, for some reason, you do not want to give the methods a "real" implementation. An `UnsupportedOperationException` is a `RunTimeException` and so is not required to be caught or declared in a `throws` clause.

```
public V put(K key, V value) (Optional)
```

Associates `key` to `value` in the map. If `key` is associated with an existing value, then the old value is overwritten and returned. Otherwise `null` is returned.

```
public void putAll(Map<? extends K,? extends V> mapToAdd) (Optional)
```

Adds all mappings of `mapToAdd` into the calling object's map.

```
public V remove (Object key) (Optional)
```

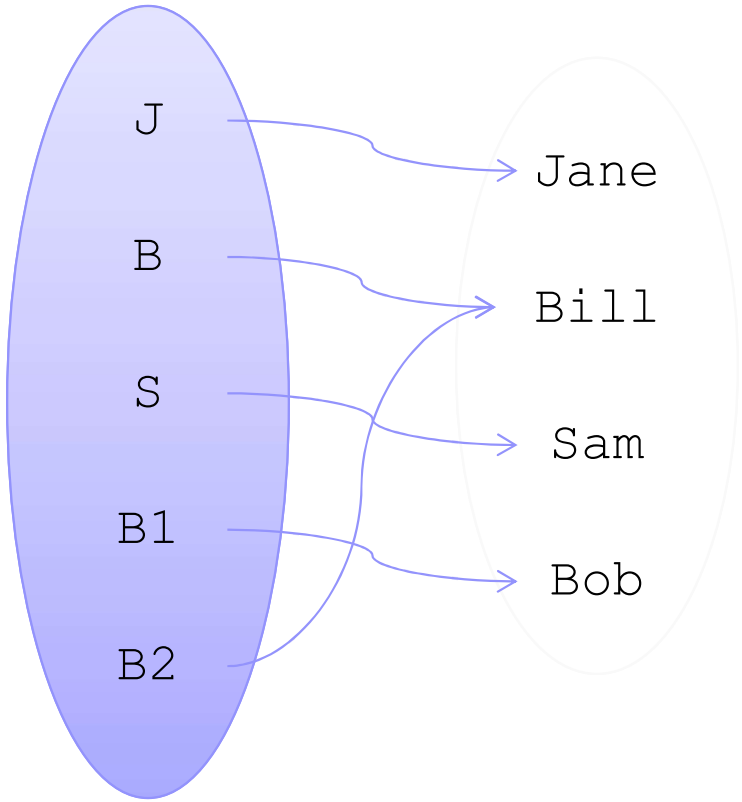
Removes the mapping for the specified key. If the key is not found in the map, then `null` is returned; otherwise the previous value for the key is returned.

Map<K,V> examples

The next command creates a **Map<K,V>** object:

```
Map<String, String> aMap = new
   HashMap<String, String>();

aMap.put("J", "Jane");
aMap.put("B", "Bill");
aMap.put("S", "Sam");
aMap.put("B1", "Bob");
aMap.put("B2", "Bill");
```

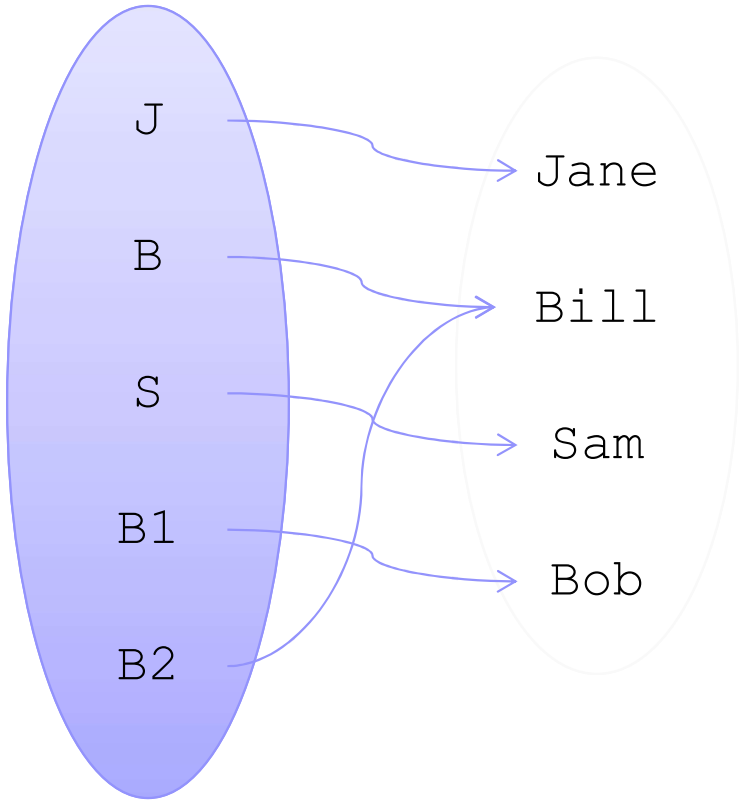


`Map<K,V>` examples

```
aMap.get("B1")
```

returns:

```
"Bob"
```



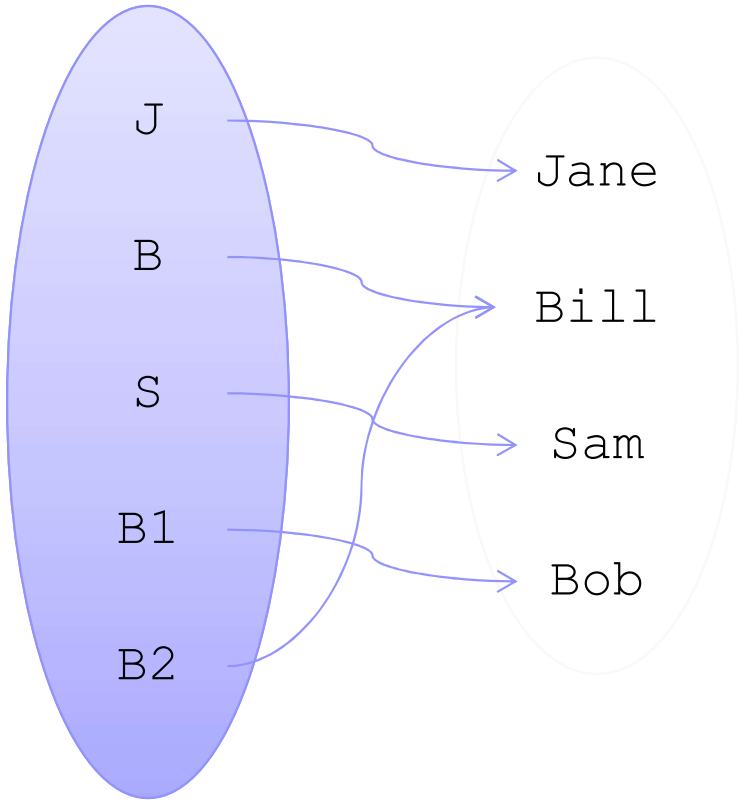
`Map<K,V>` интерфејсот

```
aMap.get("Bill")
```

returns :

```
null
```

("`Bill`" is a value, not a key)



HashMap<T>

```
public class HashMap<K,V> extends AbstractMap<K,V>
implements Map<K,V>, Cloneable, Serializable
```

This class is a concrete implementation of the `Map<K,V>` interface, utilizing a **hash table** as its underlying data structure.

- HashMap offers **constant-time (O(1)) performance** for its fundamental operations: `get` and `put` (and similarly `containsKey` and `remove`).
  - This optimal performance relies on an effective hash function that evenly distributes key-value pairs across its buckets.
- Like `HashSet`, `HashMap` **does not guarantee any specific order** for its elements (key-value pairs) and their order may change over time (e.g., during rehashing).
- It **allows one null key** and multiple null values.
- The **time complexity** for iterating over a `HashMap` is **proportional to the size** of the map (the number of key-value pairs) plus the capacity of its buckets.
- It's important to choose the initial capacity and load factor carefully. Setting the initial capacity too large (or the load factor too small) can negatively impact iteration performance.
  - By default, HashMap has an initial capacity of **16** and a load factor of **0.75**.

TreeMap<T>

```
public class TreeSet<E> extends AbstractSet<E>
implements NavigableSet<E>, Cloneable, Serializable
```

This class is another concrete implementation of the `Map<K,V>` interface (and also `NavigableMap<K,V>`), but it stores its key-value pairs in **sorted order based on the keys.**

- **The keys** in a `TreeMap` **are sorted.** This sorting is determined either by their **natural ordering** (if the key class implements `Comparable` and its `compareTo` method) or by a custom `Comparator` provided at the time of the TreeMap's construction.
- This implementation guarantees **logarithmic time complexity (O(log n))** for its fundamental operations: `containsKey`, `get`, `put`, and `remove`. This performance is achieved because `TreeMap` is typically backed by a Red-Black Tree.
- `TreeMap` does **not permit null keys,** though it does allow null values.
- As with `TreeSet`, it is critical that the equals method for the key objects is consistent with their `compareTo` method (or the provided `Comparator`).

Questions?